

系统开发工具基础

每周课上实验检查题

4 周 · 每周 4 题 · 每题 12—15 分钟

2026 年 8 月

第 1 周 2026 年 8 月 24 日 (周一, 第 5—8 节)

授课内容: 课程概览 + Shell 入门; Version Control and Git; LaTeX 文档编辑

第 1 题 含空格文件名的批量整理

主题: Shell 基础与文件系统 建议用时: 12—15 分钟

在当前目录新建 q01, 并完全使用命令行完成下列任务。

题目要求

- 创建 input/docs 和 input/tmp; 创建 input/docs/notes one.txt (内容为 alpha 和 beta 两行)、input/docs/.secret.txt (内容为 hidden)、input/tmp/empty.txt (空文件) 以及 input/run.log (任意两行日志)。
- 显示 q01 的绝对路径, 并以长格式列出 input 下全部项目, 包含隐藏文件。
- 把 input 下所有.txt 文件复制到 work/<学号>/, 保留相对目录结构并正确处理名称中的空格。
- 将复制后的目录权限设为 750、普通文件权限设为 640; 生成 inventory.txt, 列出相对路径和字节数。

第 2 题 随机访问日志统计

主题: Shell 管道与文本处理 建议用时: 12—15 分钟

在 q02 中创建 access.csv, 并使用下方给定数据完成日志统计。

给定内容

```
timestamp,user,path,status,latency_ms
09:00:01,alice,/api/users,200,120
09:00:02,bob,/api/orders,500,310
09:00:03,alice,/api/users,503,180
09:00:04,carol,/login,500,90
09:00:05,bob,/api/orders,502,260
09:00:06,dave,/health,200,20
09:00:07,alice,/api/orders,500,420
09:00:08,carol,/api/users,500,150
```

题目要求

- 编写 analyze.sh, 接收一个 CSV 文件路径; 文件不存在时将错误写入 stderr 并返回非零退出码。
- 使用 Shell 管道以及 awk、sort、head 等工具, 输出 HTTP 5xx 数量最多的前 2 个 path; 按次数降序, 次数相同时按 path 字典序。
- 输出全部数据行的平均 latency_ms, 保留两位小数; 表头不得计入。
- 分别对 access.csv 和一个不存在的文件运行脚本; 不得使用 Python、电子表格或手工统计。

第 3 题 制造、解决并解释一次合并冲突

主题：Version Control and Git 建议用时：12—15 分钟

在 q03 中从零创建一个 Git 仓库，并主动制造、解决一次合并冲突。

题目要求

- 初始化仓库，在 main 创建 config.txt，内容为 mode=normal，并完成初始提交。
- 创建 feature-a，把内容改为 mode=safe 并提交；从初始 main 创建 feature-b，把内容改为 mode=fast 并提交。
- 回到 main，先合并 feature-a，再合并 feature-b；解决冲突时保留 mode=safe，并另加一行 note=reviewed。
- 确认工作区干净，运行 git log --all --graph --decorate --oneline 查看历史。

第 4 题 修复并构建一页技术说明

主题：LaTeX 文档编辑 建议用时：12—15 分钟

将下方模板保存为 q04/report.tex，补全内容并从命令行构建 PDF。

给定内容

```
\documentclass{article}
\usepackage{amsmath}
\begin{document}
\title{Tool Report}
\author{姓名-学号}
\maketitle
\section{Result}
% 在此加入公式、表格及正文引用
\end{document}
```

题目要求

- 加入一个带编号和 label 的公式，并在正文中使用 ref 或 eqref 引用。
- 加入一个 2 行 2 列表格，设置 caption 和 label，并在正文中引用该表。
- 使用 latexmk -pdf -halt-on-error 或 tectonic 生成 report.pdf。
- 确认 PDF 能打开，且交叉引用不显示 “??”。

第 2 周 2026 年 8 月 31 日 (周一, 第 5—8 节)

授课内容：命令行环境；开发环境与工具；Debugging and Profiling

第 5 题 控制一个可清理的后台任务

主题：命令行环境：进程、信号与任务控制 建议用时：12—15 分钟

将下方脚本保存为 q05/worker.sh，再完成任务控制。

给定内容

```
#!/usr/bin/env bash
trap 'echo CLEAN_EXIT >> cleanup.log; exit 0' TERM INT
n=0
while true; do
    echo "$n"
    n=$((n+1))
    sleep 1
done
```

题目要求

- 赋予脚本执行权限，在前台启动，并把 stdout、stderr 分别写入 stdout.log、stderr.log。
- 使用 Ctrl-Z 挂起任务，再用 bg 让它在后台继续；使用 jobs 确认状态。
- 使用 jobs -p 或等价方式取得 PID，不得手工抄写 PID；发送 SIGTERM 并等待任务结束。
- 确认 cleanup.log 中出现 CLEAN_EXIT，且任务已经退出；禁止使用 kill -9。

第 6 题 语义重构与本地开发反馈

主题：开发环境与工具 建议用时：12—15 分钟

在 q06 中按照给定代码创建三个 Python 文件，并使用编辑器语言服务器完成一次语义重构。

给定内容

```
# math_utils.py
def total_price(price: float, count: int) -> float:
    return price * count

# app.py
from math_utils import total_price
print(total_price(12.5, 4))

# test_math_utils.py
from math_utils import total_price
def test_total_price():
    assert total_price(12.5, 4) == 50.0
```

题目要求

- 在已配置好 Python、pytest 和 ruff 的课程环境中打开 q06，并启用 Python 语言服务器。
- 分别演示跳转到定义和查找引用。
- 使用“重命名符号”把 total_price 改为 calculate_total，保证定义和所有代码引用同步改变。
- 在 app.py 中临时加入未使用的 import os，用 ruff 发现并删除；最后运行 ruff 和 pytest 并保证通过。

第 7 题 用调试器定位归并排序缺陷

主题：Debugging 建议用时：12—15 分钟

将下列存在缺陷的归并排序代码保存为 q07/merge_sort.py，再使用调试器定位并修复。

给定内容

```
def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i]); i += 1
        else:
            result.append(right[i]); j += 1 # 此处存在缺陷
    return result + left[i:] + right[j:]

def merge_sort(a):
    if len(a) <= 1: return a
    m = len(a) // 2
    return merge(merge_sort(a[:m]), merge_sort(a[m:]))

print(merge_sort([3, 1, 4, 1, 5, 9, 2, 6]))
```

题目要求

- 先运行给定输入，确认结果错误或出现异常。
- 使用 pdb、IDE debugger 或等价调试器，在 merge 中设置断点并观察 i、j、left[i]、right[j]。
- 完成最小修复，不得用 sorted 替换归并排序。
- 使用 pytest 增加两个测试：给定输入和含重复元素的列表；保证测试通过。

第 8 题 先测量，再优化慢速词频程序

主题：Profiling 建议用时：12—15 分钟

在 q08 中创建可重复生成的词表和故意低效的去重程序，再使用性能分析工具优化。

给定内容

```
# generate_words.py
import random
```

```
random.seed(20260831)
vocab = [f"w{i}" for i in range(1000)]
with open("words.txt", "w", encoding="utf-8") as f:
    f.write(" ".join(random.choice(vocab) for _ in range(30000)))

# wordfreq.py
words = open("words.txt", encoding="utf-8").read().split()
unique = []
for word in words:
    if word not in unique:
        unique.append(word)
print("count=", len(unique))
```

题目要求

- 生成 words.txt，使用 time 运行原程序 2 次并记录耗时。
- 使用 cProfile -s cumulative 运行一次，找出主要耗时位置。
- 在不改变最终 count 的前提下优化程序；不得写死 1000。
- 使用相同输入再运行 2 次，计算优化前后中位数和加速比。

第 3 周 2026 年 9 月 7 日 (周一, 第 5—8 节)

授课内容: Packaging and Shipping Code; 智能体编程; 不止于代码; Python 与 PyTorch

第 9 题 从源码构建并在干净环境安装 Wheel

主题: Packaging and Shipping Code 建议用时: 12—15 分钟

在 q09 中按给定内容创建一个最小 Python 命令行包, 并验证 wheel 安装。

给定内容

```
# src/greetlab/cli.py
import argparse
def main():
    p = argparse.ArgumentParser()
    p.add_argument("--name", required=True)
    a = p.parse_args()
    print(f"Hello, {a.name}!")

# pyproject.toml
[build-system]
requires = ["setuptools>=68"]
build-backend = "setuptools.build_meta"
[project]
name = "greetlab-学号"
version = "0.1.0"
[project.scripts]
sdt-greet = "greetlab.cli:main"
```

题目要求

- 创建 src/greetlab/__init__.py、给定的 cli.py 和 pyproject.toml, 并把“学号”替换为自己的学号。
- 在已安装 build 工具的课程环境中运行 python -m build, 生成 wheel。
- 新建一个干净虚拟环境, 只从生成的 wheel 安装。
- 切换到 q09 之外运行 sdt-greet --name <学号>, 确认输出正确。

第 10 题 让编程智能体进入可验证的修复循环

主题: 智能体编程 建议用时: 12—15 分钟

复用第 9 题的 greetlab 包, 使用编程智能体完成一个小型测试驱动修复。

题目要求

- 复制 q09 为 q10, 添加一个失败测试: 当 name 只含空白字符时, main 应以 SystemExit(2)结束。
- 先运行测试确认失败, 再向编程智能体说明目标、约束和测试命令, 要求它修改实现并运行测试。
- 人工检查 diff, 撤销无关修改, 并再次运行测试。

- 在 `ai_log.md` 中用不超过 5 行记录核心提示、智能体改动和人工验证。

第 11 题 把“无法处理”的协作材料改成可执行信息

主题：不止于代码 建议用时：12—15 分钟

围绕“空姓名仍输出问候语”这一缺陷，改写下面三段质量较差的协作材料。

给定内容

Issue: Windows 上运行不了，尽快修复。
提交信息: `fix bug`
评审意见: 这里写得不好，重写。
已知事实: 运行 `sdt-greet --name " "` 时仍输出 `Hello, !`，并以 `0` 退出。

题目要求

- 在 `communication.md` 中重写 Issue，包含环境、复现命令、期望结果和实际结果；未知信息写“待确认”。
- 重写提交信息：标题使用祈使语气，正文说明问题和解决方案。
- 重写评审意见：指出具体行为、风险和建议动作，并标注 Blocking、Suggestion 或 Nit。
- 全文控制在 400 字以内，保持专业、简洁。

第 12 题 修复一个可复现的线性回归训练循环

主题：Python 与 PyTorch 建议用时：12—15 分钟

将下方代码保存为 `q12/train.py`，补全 TODO 并完成一次 PyTorch CPU 训练。

给定内容

```
import torch
from torch import nn
torch.manual_seed(20260907)
x = torch.linspace(-1, 1, 101).reshape(-1, 1)
y = 3 * x - 1
model = nn.Linear(1, 1)
loss_fn = nn.MSELoss()
opt = torch.optim.SGD(model.parameters(), lr=0.1)
for _ in range(200):
    pred = model(x)
    loss = loss_fn(pred, y)
    # TODO: 清空梯度、反向传播、更新参数
# TODO: 进入评估模式并在 no_grad 中打印最终损失、weight 和 bias
```

题目要求

- 补全训练循环，正确调用 `zero_grad`、`backward` 和 `step`。
- 训练后调用 `model.eval()`，并在 `torch.no_grad()` 中计算最终损失。

- 打印最终损失、weight 和 bias; 最终损失应小于 0.001。
- 不得直接把 weight 和 bias 赋值为 3 和 -1。

第 4 周 2026 年 9 月 14 日 (周一, 第 5—8 节)

授课内容：代码质量；元编程；大杂烩；课堂综合测试

第 13 题 建立可执行的本地质量门禁

主题：代码质量 建议用时：12—15 分钟

复用第 10 题完成的 greetlab 包，在 q13 中建立一个轻量本地质量检查。

题目要求

- 复制 q10 为 q13；在 pyproject.toml 中加入 ruff 和 pytest 的最小配置。
- 补充一个正常姓名测试和一个空白姓名测试，每个测试包含有效断言。
- 运行 ruff format、ruff check 和 pytest，修复全部问题，不得全局忽略规则。
- 编写 check.sh，依次执行 ruff format --check、ruff check 和 pytest；运行并保证返回 0。

第 14 题 让 Make 只重建真正受影响的产物

主题：元编程：构建系统 建议用时：12—15 分钟

在 q14 中按给定内容创建三个源文件，只需完成 Makefile。

给定内容

```
# data.csv
name,value
alpha,2
beta,3
gamma,5

# stats.py
import csv
with open("data.csv") as f:
    total = sum(int(r["value"]) for r in csv.DictReader(f))
open("stats.txt", "w").write(str(total))

# build_report.py
text = open("report.md").read()
total = open("stats.txt").read()
open("report.txt", "w").write(f"{text}\nTotal: {total}\n")

# report.md
# Data Report
```

题目要求

- 创建给定的 data.csv、stats.py、build_report.py 和 report.md。

- 编写 Makefile，包含 all、stats.txt、report.txt 和 clean；准确列出数据文件、脚本和中间产物依赖，并把 all、clean 声明为.PHONY。
- 验证首次 make 生成两个产物；第二次无改动时不执行配方。
- touch data.csv 后再次 make，确认 stats.txt 和 report.txt 重新生成；clean 只删除生成物。

第 15 题 把本地 API 数据转换为可读报告

主题：大杂烩：API、jq、CLI 约定与 Markdown 建议用时：12—15 分钟

在 q15 中创建给定 JSON 和本地 HTTP 服务，再用 curl 与 jq 生成简短 Markdown 报告。

给定内容

```
[
  {"name": "alpha", "status": "active", "downloads": 120, "version": "1.2.0"},
  {"name": "beta", "status": "inactive", "downloads": 900, "version": "2.0.0"},
  {"name": "gamma", "status": "active", "downloads": 450, "version": "1.5.1"},
  {"name": "delta", "status": "active", "downloads": 450, "version": "0.9.0"},
  {"name": "epsilon", "status": "active", "downloads": 80, "version": "3.1.0"}
]
```

题目要求

- 把给定 JSON 保存为 packages.json，并在 q15 目录运行 python -m http.server 8000。
- 使用 curl -fsS 获取 http://127.0.0.1:8000/packages.json。
- 使用 jq 筛选 status 为 active 且 downloads 不少于 100 的记录，并按 downloads 降序、name 升序排列。
- 编写 api_report.sh，把结果生成 summary.md，包含标题和 name/version/downloads 三列表格。

第 16 题 修复并交付一个陌生的小型工具仓库

主题：课堂综合测试 建议用时：12—15 分钟

复用第 13 题的项目，在 q16 中完成一次 15 分钟综合检查。

题目要求

- 复制 q13 为 q16 并初始化/继续使用 Git。
- 把问候语实现临时改成始终返回字面量 Hello, name!，运行 check.sh 确认测试失败；随后定位并修复。
- 编写最小 Makefile，包含 check、build 和 clean；check 调用 check.sh，build 生成 wheel。
- 运行 make check 和 make build，计算 wheel 的 SHA-256，并把最终改动提交为一个内容聚焦的 Git 提交。